

Overview

The replication data for this paper is made up of 5 CSV files and 2 JSON files. The table summarizes their details:

File Name	Format	Details
python_fix_prs.csv	CSV	contains the metadata for every fix-type pull request in the dataset. Each row is one PR and records its ID, number, title, description body, which AI agent authored it, the GitHub username and user ID, the PR state, the dates it was created, closed, and merged, the repository ID and URL, and the direct link to the PR on GitHub.
python_fix_prs_with_stats.csv	CSV	a reduced version of the file above, keeping only the 1,211 PRs that are written in Python and were successfully merged. It adds four columns pulled from the GitHub API: how many lines were added, how many were deleted, how many files were changed, and the total lines of code changed. These numbers are used to normalize issue counts by code size.
All_PR_Sonar_Results.json	JSON	holds the raw output from running SonarQube on each of the 1,210 PRs. For each PR, it records the repository owner, name, agent, the commit hash before the merge, and the commit hash after the merge. It also stores the total number of issues found and the full details of each issue, including the rule that was violated, the severity level, the issue type, the diagnostic message, the file and line number where it was found, the estimated fix effort, the technical debt value, and any associated tags.
All_PR_Sonar_Results.csv	CSV	is the same data as the JSON file above but stored in CSV format. Each row is one PR. The list of issues for that PR is stored as a single text string in one column rather than as structured nested data.
All_PR_Issues_Details_with_LOC.csv	CSV	breaks the SonarQube results down so that each row is one individual issue rather than one PR. It records the issue type, severity, diagnostic message, the file name, the start and end line numbers, and the rule ID. It also includes the LOC statistics from python_fix_prs_with_stats.csv for the PR that issue belongs to. This is the file used directly to produce the results for both RQ1 and RQ2.
Security_Hotspots.json	JSON	stores the SonarQube security hotspot results separately from the regular issues, because SonarQube retrieves hotspots through a different API endpoint. For each PR with at least one hotspot, it records the hotspot's unique key, its security category, its risk level (LOW, MEDIUM, or HIGH), its

File Name	Format	Details
		review status, the diagnostic message, the file and line location, and the author and timestamp information.
Security_Hotspots_Details.csv	CSV	the flattened version of the JSON file above, with one row per individual hotspot. Each row records the repository, PR number, agent, GitHub URL, issue type, severity, message, file name, start and end lines, and the security rule category. This file is used for the hotspot-specific portions of the analysis tables.

Of the 7 files in the replication package, only “python_fix_prs.csv” originates directly from the AIDev dataset, the rest were added by the authors for the paper.

PR 1

URL: <https://github.com/marimo-team/marimo/pull/4035>

Is it a bug fix? Not exactly. Devin had a solution that the agent seemed to think had fixed the issue but upon further reading of the conversation that happened at the original issue. The user was dissatisfied with the fix Devin put ahead, so they add their own fix.

The bug: When a user loaded a wide DataFrame (100+ columns) into the data editor component, no minimum column width was enforced. The columns would be rendered too narrow to read the header names.

The fix: Devin added a min-width CSS property to the DataEditorPlugin column definitions, ensuring all columns have at least enough width to display their header text.

Author check: The dataset records the agent as Devin and the user as devin-ai-integration[bot]. This is correct — the PR was opened by the Devin bot account.

PR 2

URL: <https://github.com/crewAIInc/crewAI/pull/2548>

Is it a bug fix? Yes.

The bug: The classes TaskOutput and CrewOutput were referenced in the official crewAI documentation but were not included in the package's __init__.py public exports. Any user who followed the documentation and tried to import these classes directly would receive an ImportError at runtime.

The fix: Devin added the missing import statements for both TaskOutput and CrewOutput to `__init__.py`, making them properly accessible as part of the public API. The change touched only the exports file.

Author check: The dataset records the agent as Devin and the user as devin-ai-integration[bot]. This is correct.

PR 3

URL: <https://github.com/reflex-dev/reflex/pull/5616>

Is it a bug fix? Yes. The PR references an internal issue ticket (ENG-6818) and the body describes a concrete runtime defect with a specific root cause.

The bug: When a cookie value happened to contain a JSON-formatted string (e.g. `{"key": "value"}`), the universal-cookies JavaScript library was silently converting it from a string into a JavaScript dictionary object during the `cookies.get()` call. Reflex's backend expected all cookie values to arrive as strings. When they arrived as objects instead, this caused type validation errors at runtime. The problem was invisible to developers because the cookie appeared to be set correctly — the issue only surfaced when its value happened to look like JSON.

The fix: Devin added `true` as the second argument to both `cookies.get()` calls in `reflex/.templates/web/utils/state.js`. In the universal-cookies library, this flag is the `doNotParse` option, which disables automatic JSON parsing and ensures cookie values are always returned as raw strings regardless of their content. The change was two lines in a single file, with test coverage added alongside it.

Author check: The dataset records the agent as Devin and the user as devin-ai-integration[bot]. This is correct — the PR was opened entirely by the Devin bot account with no human user involved.